mb-free___faults-shrink

An AgentMPI run analysis

Abstract

This is the `SHRINK` point of the four-policy barrier-failure sweep in `bench_faults` (`experiments/microbench.py`): eight ranks, rank 3 returns immediately to simulate a crashed agent session, and the seven survivors enter a barrier with a 15 s deadline and the policy under test. Of the four policies it is the only one that runs the whole fault-tolerance stack — detection, declaration, in-place shrink and then a fault-tolerant `agree` — and it is the only one whose recovery path visibly breaks. Detection works exactly as designed: all seven survivors time out within 2.7 ms of each other at $t = 15.198$–15.201 s, all seven name `absent: [3]`, and the benchmark records `detected_correctly: true` with a median detection latency of 15.196 s against a 15.0 s deadline. What follows is a cascade. The `agree` that the harness calls next never reaches quorum, blocks for its full 60 s timeout, and terminates with seven *spurious* `ft.declare_failed` events naming six healthy ranks, two `ft.agree_timeout` events, and five ranks returning `result: true` with `voters: []` — agreement over an empty electorate. The cause is in the fabric and is unambiguous: `ft.shrink_in_place` executes `UPDATE comms SET generation = generation + 1` once per calling rank, so seven independent calls drove the communicator's generation from 0 to 7, and because `Communicator._record_collective` keys a collective on `(ctx, generation, epoch, op)` the seven ranks then voted into *six different collectives* (`cid` 8–13). The single most important thing to take away is that this is a genuine AgentMPI defect and not an artefact of the benchmark: `shrink_in_place` is a collective operation implemented with a non-idempotent per-rank increment, where its out-of-place sibling `ft.shrink` correctly writes an absolute generation behind a leader election. Everything downstream of the barrier in this trace is that one line's consequence.

1 What this run is

property	value
run	`mb-free__faults-shrink`
experiment	–
campaign label	–
declared world size	8
ranks appearing in the log	8
executors	`none` × 8
events	60
wall time	75.43 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.00×	total busy time over wall time
parallel efficiency	0.0%	achieved parallelism per rank
idle fraction	100.0%	rank-seconds paid for and unused
peak ranks busy	0	of 8 in the world
serial fraction of active time	0.0%	time with exactly one rank busy
load imbalance	0.00×	slowest rank's busy time over the mean
coordination share	17.6%	rank-seconds blocked in collectives, of those available
coordination in flight	20.2%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	0	invocations that reached a model
agent latency p50 / p95	0.00 / 0.00 s	per invocation
messages	0	0 eager, 0 rendezvous
tokens sent	0	0 deferred under rendezvous
tokens in / out	0 / 0	prompt and completion
cost	$0.0000	0.0000 $/kTok out

3 What the analysis notices

- [warning] `barrier/central` (label `phase`) was reached by 7 of 8 ranks, so it never completed.
- [warning] The coordination figures count time inside *completed* collectives only, and this run has ranks that never completed one. A rank records a collective on completion, so a rank that blocked and then timed out contributes nothing — the reported share is a floor, and on a badly degraded run a very loose one.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.00	0.0	0	0	0	0	0	0.0	finalized
1	0.00	0.0	0	0	0	0	0	0.0	finalized
2	0.00	0.0	0	0	0	0	0	0.0	finalized
3	0.00	0.0	0	0	0	0	0	0.0	finalized
4	0.00	0.0	0	0	0	0	0	0.0	finalized
5	0.00	0.0	0	0	0	0	0	0.0	finalized
6	0.00	0.0	0	0	0	0	0	0.0	finalized
7	0.00	0.0	0	0	0	0	0	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank's own lifetime, so a rank that joined late is not penalised for the time before it existed.

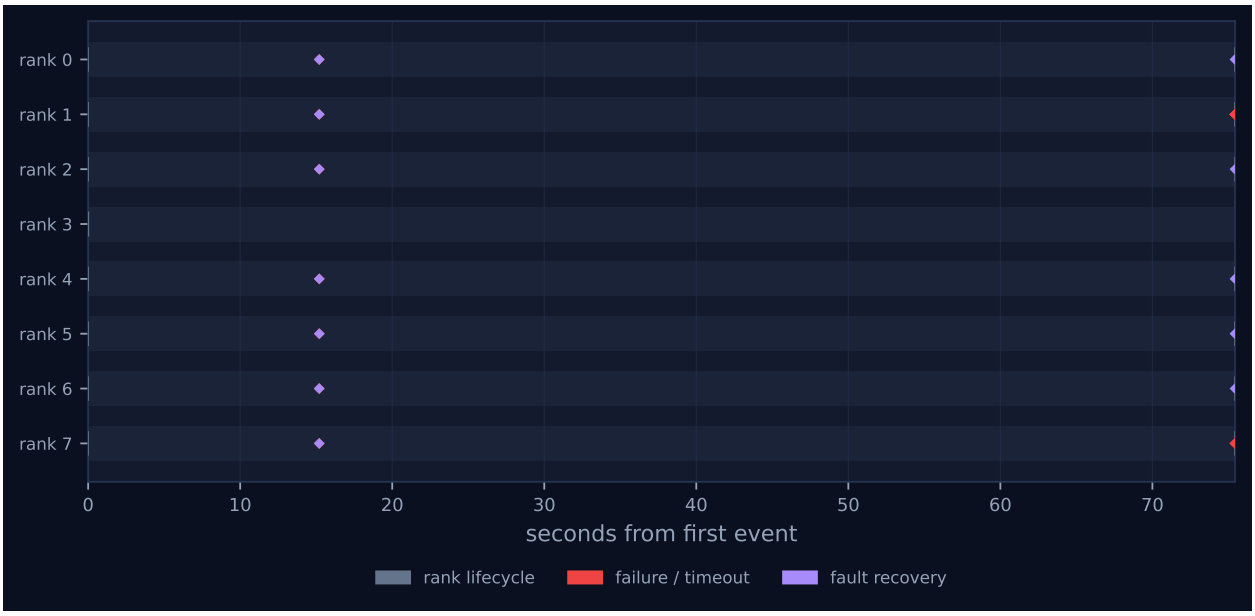

Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
barrier	central	phase	8	7	0	2	0	0	0	15.20	0.01	–

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 has eight lanes, no bars, and marks in exactly two vertical columns: one at $t \approx 15.2$ s and one at $t \approx 75.4$ s, separated by sixty seconds of nothing. There are no blue work bars anywhere because there is no work anywhere. `metrics.json` records `executors: {none: 8}`, zero agent calls, zero tokens and $0.0000, so the headline table's *achieved parallelism* 0.00× and *idle fraction 100.0%* are statements about the benchmark's design rather than findings about the protocol. `bench_faults` deliberately contains no agent invocation: its `rank_main` calls `comm.barrier` and nothing else. A busy fraction cannot be anything but zero when no rank was ever given a task, and the two rows should be read as "not applicable" rather than as a utilisation result. The same caveat covers the wall time. Every duration in this run is a timeout constant: 15 s is `-detect-timeout` from `results/microbench/free-faults.json`, 60 s is the literal in the harness's `ampi.agree(comm, True, timeout=60)` call, and 75.431 s is their sum plus about 0.4 s of poll granularity and fabric write latency.

Rank 3's lane is empty apart from its lifecycle ticks at the far left. It initialised at $t = 0.012$ s and finalised at $t = 0.013$ s, one millisecond later, which is the whole of the injected fault: `rank_main` returns `{"role": "victim"}` without touching the communicator. Nothing in the trace distinguishes this from an agent session that died before its first call, which is the point of the injection.

The first column is the detection, and it is remarkably tight. Seven `barrier.timeout` events land between 15.198 s and 15.201 s — a spread of 2.7 ms across seven independently polling processes — each carrying `absent: [3]` and `policy: "shrink"`. Each is followed within a millisecond by `ft.declare_failed`, then `ft.shrink_in_place`, then `coll.barrier`, the four-event signature repeated seven times. The tightness is worth a sentence because it is not luck. The central barrier polls the fabric with a back-off that starts at 20 ms and saturates at 250 ms, and the seven survivors entered the barrier within 6 ms of each other (`rank.init` from 0.002 s to 0.008 s), so all seven were in the same 250 ms poll phase and crossed their deadlines together. The collective table records the consequence as *skew 0.01 s*, against 0.02 s for the `PROCEED` sibling — the only other run in the sweep that logs a collective at all. The viewer renders these seven marks in violet, the recovery colour, and its legend counts *fault recovery 12* — the seven `ft.shrink_in_place` events plus the five `ft.agree` events sixty seconds later.

The second column is where the run goes wrong, and the viewer shows it more clearly than any number does. At $t \approx 75.4$ s, five lanes (ranks 0, 2, 4, 5, 6) carry a violet diamond and two lanes (ranks 1 and 7) carry a *red* one. That colour split is the finding: seven ranks entered the same agreement and two of them left it by raising while five left it by returning. Under `ft.agree`'s own docstring — "either every surviving caller returns the same value, or all of them raise" — those two colours should not appear in the same column.

The sixty-second gap between the columns contains no events at all, and that absence is itself a measurement. All seven survivors were inside `ft.agree`, polling the `coll_parts` table on the same 20 ms-to-250 ms back-off. Reproducing that schedule arithmetically gives about 250 polls per rank over 60 s, so roughly 1,750 `SELECT` statements crossed the fabric in a window the trace renders as empty; the barrier's own 15 s wait contributed about 490 more. This is what the run's wall clock is actually made of, and it is why nothing here should be read as a cost measurement for anything but SQLite.

The viewer's FAILURES tile reads **14** while its rank table shows all eight ranks `finalized` with `suspected: -`, and `metrics.json` reports `failed_ranks: []` beside `summary.failures: 14`. These are not in conflict; they count different things. `failures` counts `ft.declare_failed` events in the log, and `failed_ranks` is copied verbatim from the `job.finish` payload, which lists ranks whose `rank_main` raised. The benchmark catches `ampi.AmpiError` inside `rank_main`, so no rank propagated an exception to the launcher, and the job reports `ok: true` for a run in which the fabric ended up holding eight failure rows and seven of the eight world ranks were at some point declared dead. The rank table's `suspected: -` column is consistent for the same reason: it reflects the lease-based detector in `ft.health`, which was never consulted, not the `failures` table.

7 Interpretation

7.1 What is true by construction

The shape of the first fifteen seconds is entirely dictated by the harness and the policy, and none of it is a discovery. `bench_faults` chose $p = 8$ and victim set `[3]`; `algorithms.barrier` forces

`algorithm = "central"` whenever the policy is `PROCEED`, `SHRINK` or `REVOKE`, because only the arrival-counting algorithm can name absentees; and the central barrier therefore sends no messages at all, which is why `metrics.json` reports `messages: 0` and `eager_messages: 0` for a run whose entire purpose was a collective. That zero is a property of the algorithm selection, not evidence of a quiet run: the barrier's coordination went through 490-odd fabric reads instead of through the message layer, and none of that traffic is visible in the message counters. Likewise the barrier's `rounds: 0` against `predicted 2` in the collective table is a logging gap rather than a disagreement — `algorithms.barrier` sets a local `rounds = 2` for the central path and returns it in `BarrierResult`, but never writes it onto the `_Tracker`'s `CollStats`, so the emitted `coll.barrier` always carries zero. The tooling correctly declines to score this as a model failure (– in the model column).

7.2 The mechanism, from the trace

The policy's realisation is eleven lines of `_central_barrier` in `src/agentmpi/algorithms.py` and the trace matches them one for one. Each survivor registers its arrival in `coll_parts` through `_record_collective`, then polls `SELECT crank FROM coll_parts WHERE cid=?` until either the arrival count reaches `comm.size` or the deadline passes. At the deadline it computes `absent = {0..7} - arrived`, emits `barrier.timeout`, and — because the policy is neither `RAISE` nor `WAIT` — calls `ft.declare_failed` for every absentee, then `ft.shrink_in_place`, then returns the absentee tuple normally rather than raising. That last detail is what makes `SHRINK` and `PROCEED` the only two policies in the sweep for which the benchmark records `all_completed: true`: the barrier hands the harness a `BarrierResult` whose `absent` field is `(3,)` and whose `complete` property is false, and the harness treats a returned result as success.

Why fourteen `ft.declare_failed` events for one failure

The fourteen split cleanly into seven plus seven, and only the first seven are about rank 3.

The first seven are the barrier's, all at $t = 15.198$–15.204 s, all with `failed: 3` and `detail: "barrier phase"`. There are seven because `declare_failed` is called by each rank independently — the central barrier has no leader, so every survivor discovers the same absentee and records it. The database deduplicates and the log does not: the `INSERT` into `failures` carries `ON CONFLICT(ctx, rank, kind) DO NOTHING`, and querying the run's fabric (`runs/mb-free/faults-shrink/fabric.sqlite`) shows a single row for rank 3, while `fabric.emit` sits outside the conflict clause and fires unconditionally. So the event count is the number of *observers*, and the row count is the number of *failures*. Seven observers, one failure. This is defensible — an event log should record who noticed what and when, not a deduplicated summary — but it means the viewer's FAILURES tile of 14 and `metrics.json`'s `summary.failures: 14` both overstate the number of dead ranks by an order of magnitude, and neither artefact says so.

The second seven are not about rank 3 at all. At $t = 75.424$ s rank 1 emits six `ft.declare_failed` events inside a millisecond, naming ranks 0, 2, 4, 5, 6 and 7 with `detail: "agree timeout"`, and at $t = 75.425$ s rank 7 emits one naming rank 1. All seven name a rank that was alive, healthy, and at that moment sitting in the same `agree` call. They come from the timeout branch of `ft.agree`, which declares every non-voter failed before raising. By the end of the run the `failures` table holds eight rows: rank 3, correctly, and every other rank in the world, incorrectly.

Why two `ft.agree_timeout` events, and only two

The two are rank 1's and rank 7's, 1.1 ms apart, and the reason there are exactly two rather than seven is a race that the first timeout sets up for the others.

`ft.agree`'s loop computes `expected = {c for c in range(comm.size) if c not in get_failed(comm)}` on every iteration and completes as soon as `expected` is a subset of the ranks that have voted. Rank 1 reaches its deadline first. At that instant the `failures` table holds only rank 3, so its `expected` is the seven survivors, it has seen one vote (its own), and its `missing` list is the other six: `ft.agree_timeout` with `missing: [0, 2, 4, 5, 6, 7]`, preceded by six declarations. Rank 7 reaches its deadline 1 ms later, and by then those six declarations are in the table, so *its* `expected` has collapsed to `{1}` alone: `missing: [1]`, one declaration, one `ft.agree_timeout`. Those two events between them have now declared all eight world ranks failed.

The remaining five ranks — 0, 2, 4, 5 and 6 — never reach their deadlines. On their next poll, within five milliseconds, they recompute `expected` against a `failures` table that now contains everybody, get the empty set, find that the empty set is trivially a subset of what has voted, and take the success branch. Python's `all()` over an empty generator is `True`, so all five return `True` and emit `ft.agree` with `result: true`, `voters: []`, `excluded: [0, 1, 2, 3, 4, 5, 6, 7]`. Five ranks agreed that a proposition held, unanimously, among nobody.

7.3 The root cause: a collective operation implemented as a per-rank increment

None of the above should have happened, because all seven survivors called `agree` with the same value at nearly the same instant. Querying the fabric shows why they never saw each other:

`cid`	`generation`	`epoch`	voters recorded in `coll_parts`	
1	0	1	0, 1, 2, 4, 5, 6, 7	(the barrier)
8	2	2	7	
9	3	2	0	
10	4	2	2	
11	5	2	1	
12	6	2	4	
13	7	2	5, 6	

Seven ranks, one `agree`, six collectives. Five ranks voted entirely alone and ranks 5 and 6 happened to pair up. No `cid` could ever have reached a quorum of seven, so the 60 s block was inevitable from the moment the votes were cast.

The mechanism is one line. `ft.shrink_in_place` ends with

```
cur.execute("UPDATE comms SET generation = generation + 1 WHERE ctx=?",
                                (comm.ctx,))
```

followed by `comm.refresh()`, which reads the generation back into the caller's cache. Seven ranks called it, so the counter was incremented seven times and the run's `comms` row ends at `generation = 7`. Each rank refreshed at a different point in that sequence and cached a different value — rank 7 read 2, rank 0 read 3, rank 2 read 4, rank 1 read 5, rank 4 read 6, ranks 5 and 6 both read 7. Then `_record_collective` keyed the `agree` on `(ctx, generation, epoch, op)`, and six distinct generations produced six distinct `cid`s.

I would call this a real defect in AgentMPI rather than expected behaviour, for three reasons. First, the increment is relative where every other generation write in the codebase is absolute: the out-of-place `ft.shrink` elects a leader, has that leader alone create the new context, publishes the result through a content-keyed `meta` row so that non-leaders converge on the same `ctx`, and writes `UPDATE comms SET generation=?` with the value `comm.generation + 1`. Run seven times concurrently, that code converges; `shrink_in_place` does not. Second, the operation is unavoidably collective. It is called from inside `_central_barrier`, which every participant executes, so "seven ranks call it at once" is the only way it is ever invoked, not an unusual usage. Third, the blast radius is wider than the one symptom seen here: `Communicator._itag` embeds `self.generation` in the internal tag of every point-to-point message a collective sends, so ranks left holding different generations cannot match each other's messages either. This run only exercised the fabric-keyed path; a message-passing collective after an in-place shrink would fail the same way, and less legibly.

The fix is small and local — compute the target generation once and write it absolutely, as `shrink` already does, or make the increment conditional on the current value — but I would not change the code on the strength of this trace alone without also checking whether any caller depends on the counter advancing once per participant.

7.4 A second defect: agreement over an empty electorate

The vacuous `True` is a separate bug from the generation split, and it would remain after the split was fixed. `ft.agree`'s completion test is `if expected <= set(voted)`, with `result = all(v for c, v in voted.items() if c in expected)`. When `expected` is empty both are trivially satisfied, so the function returns `True` without a single vote having been counted, and reports it in the trace as `voters: []`. The same module already knows this state is pathological and says so elsewhere: `_agree_survivors` raises `AmpiProcFailed("no survivors")` when the survivor set empties out. `agree` should do the same, and a caller receiving `True` from a consensus primitive is entitled to assume somebody consented.

The consequence in this run is the contract violation visible in the timeline's colour split. `ft.agree`'s docstring promises that "either every surviving caller returns the same value, or all of them raise". Here two callers raised `AmpiProcFailed` and five returned `True`, and the five were not a superset or subset of any coherent survivor set — by their own emitted payload they excluded themselves. This is exactly the split-brain outcome that fault-tolerant agreement exists to prevent, produced by the agreement primitive itself.

It is worth being clear about what the benchmark says about this, because it says the wrong thing. `results/microbench/free-faults.json` records `"agree_consistent": true` for the shrink row. That field is computed as `len({s.get("agreed") for s in survivors if s.get("agreed") is not None}) <= 1`, and the two ranks that raised recorded `agreed = None` because `bench_faults` catches `ampi.AmpiError` and leaves the field unset. The metric therefore filters out precisely the ranks whose disagreement it exists to detect, and reports consistency across the five that survived to answer. The published row is misleading and the trace is not; anyone reading the fault table in the paper should be told that the shrink row's `agree_consistent` is vacuous here.

7.5 A third, quieter defect: in-place shrink does not shrink anything a collective reads

Even had the generation counter behaved, the repair this policy performs is largely nominal. `shrink_in_place` sets `comm_members.state = 'failed'` for the absentees, and the run's fabric confirms it: `crank 3` ends the run marked `failed` and the other seven `active`. `Communicator.refresh` duly reads that column into a `_active` tuple. But `_active` is *written and never read* — it appears exactly once in `src/agentmpi`, at the assignment. `Communicator.size` is `len(self._members)`, which counts failed members too, and `_central_barrier`'s completion test is `len(arrived) >= comm.size`. So a second barrier on this communicator would wait the full timeout for rank 3 all over again, which is precisely what the docstring's "only excludes the dead from future collectives" says it will not do. The one operation that does exclude the dead, `agree`, gets there by a different route entirely: it consults the `failures` table through `get_failed`, not the member states.

The benchmark cannot catch this, because `rank_main` runs exactly one barrier. That makes the finding inferential from the source rather than measured from this trace, and I flag it as such — but the inference is short, and the trace does supply the two halves of it: the member row is marked `failed`, and `comm.size` is still reported as 8 in the `coll.barrier` payload emitted after the shrink.

7.6 What `ft.agree` is for, and why it is needed here at all

It is worth stating plainly, because the name invites the wrong reading: `agree` repairs nothing. It is fault-tolerant consensus on a boolean over a group whose membership is itself in doubt, and its output is a decision, not a communicator. The `ft.agree` payload's two lists say exactly that — `voters` is the set whose consent was required and obtained, `excluded` is the set written off — and in a healthy run they partition the world.

The reason a decision is needed after a failure is that shrinking leaves every survivor holding a *local* answer to a *global* question. Rank 5 knows that its own barrier timed out and that it saw rank 3 absent; it does not know whether rank 6 saw the same absentee set, whether rank 6 also shrank, or whether the phase whose completion is now in question actually completed at rank 6. Without agreement two survivors can proceed on incompatible premises — one rebuilding a seven-member group and another a six-member one, or one treating the phase as done and another retrying it — and because the group has already lost a member, an ordinary `allreduce` over logical AND cannot be used to reconcile them: it would block forever on the dead rank. That is the gap `agree` fills, and it is why `ft.shrink` calls `_agree_survivors` before building anything. AgentMPI is explicit that this is not a circumvention of FLP: the vote is routed through the fabric, which is a shared fate-sharing point, and the docstring says so rather than claiming otherwise.

This run is therefore a demonstration of why the primitive matters, delivered backwards. The seven survivors did diverge — into six different views of the communicator's generation — and `agree` was the operation that would have caught it. It failed to, because the divergence was in the very key it uses to find its peers.

7.7 The flagged findings

Two findings are generated for this run and both are expected rather than defects.

"*`barrier/central` (label `phase`) was reached by 7 of 8 ranks, so it never completed.*" This is the experiment. The benchmark kills rank 3 on purpose so that the barrier cannot complete, and the

policy under test is defined entirely by what happens at that moment. A run of this benchmark in which the barrier *did* complete would be the defective one. The finding is also the reason the collective table shows – in the model column and `model_checks_total` is 0: `analysis.py` gates the cost-model comparison on `n_participants >= size`, correctly declining to score a closed form against an invocation that did not happen.

"*The coordination figures count time inside* completed *collectives only… the reported share is a floor.*" Also expected, and on this run the floor is unusually loose in a way the sibling runs make visible. The numerator behind *coordination share 17.6%* is 106.367 rank-seconds, which is 7×15.195 s: the barrier waits, and nothing else. The 60 s that all seven survivors then spent blocked inside `ft.agree` — 420 rank-seconds, four times the recorded figure — contributes nothing, because `ft.agree` is not a `coll.*` event and is not counted as a collective at all. Worse, that unrecorded blocking is in the *denominator*: coordination share is defined against `world_size` × wall time, which the extra sixty seconds inflates from 121.7 to 603.4 rank-seconds. So this run reports **17.6%** coordination and its `proceed` sibling reports **87.4%**, with an identical numerator, purely because this run coordinated for sixty seconds longer in a way the metric cannot see. Reconstructed from the log, $7 \times (15.2 + 60.2) = 527.8$ rank-seconds of the 603.4 available were spent blocked in coordination, or 87%. The metric is correct as defined; the definition is not the quantity a reader of this run wants, and `coordination_is_underreported: true` is the right disclosure.

7.8 Against its siblings, and what a harness author should do

Across the four policies the trace supports a clear ordering, and it is not the one the names suggest.

`WAIT` and `RAISE` are indistinguishable from each other and diagnostically dead. Both leave `algorithm` at its `dissemination` default — only `PROCEED`, `SHRINK` and `REVOKE` switch to `central` — and the dissemination barrier cannot name absentees, so neither run contains a single `barrier.timeout`, `ft.*` or `coll.*` event. Their fabrics' failure tables are empty. Both fail with `ERR_TIMEOUT: receive timed out [...source=4 ...]`, naming a live rank rather than the dead one, and the benchmark scores both `detected_correctly: false`.

`PROCEED` detects correctly, in 15.195 s, at a cost of one failure row and seven events, and leaves the communicator untouched — generation 0, all eight members `active`. It buys knowledge and defers the repair.

`SHRINK` detects identically (15.196 s) and then attempts the repair, and in this trace the repair is what breaks. It costs an extra 60.2 s of wall clock, seven false failure declarations, an inconsistent agreement, and a communicator whose generation counter is seven ahead of where any rank thinks it is.

The recommendation is therefore `PROCEED` for phases whose input can be partial, and `REVOKE` followed by an explicit out-of-place `ft.shrink` for phases whose input cannot be — and not the `SHRINK` barrier policy, until the generation increment is fixed. `REVOKE` is untested in this sweep, which is a real gap in the evidence, but it is the only escalating policy whose implementation does not route through `shrink_in_place`: it calls `ft.revoke`, which writes an absolute `revoked=1` and is idempotent by construction, and then raises, leaving the harness to call `ft.shrink` — the sibling that does elect a leader and does converge.

The counterexample that makes this concrete is elsewhere in the archive. `real-tr-p16-full` is the only genuine failure among the 500 runs: sixteen ranks, all sixteen in `failed_ranks`, `ok: false`, 7,200 s of wall clock, $0.0726 spent and no output. Six rank roles were never bound to a worker

process, and the ten that worked correctly died waiting inside a glossary `allreduce` that could never fold. That run's harness imports none of `revoke`, `shrink` or `agree`; its `failures` table has zero rows and its log has zero `ft.declare_failed` events. The present run's first fifteen seconds are precisely the affordance it lacked — name the absentee, record the failure, hand the harness a list — and its own analysis concludes that a shrink at $t \approx 190$ s over the ten ranks that actually had executors would have finished the job in minutes. (Ten, not twelve: twelve is the number of ranks that emitted a `coll.reduce` record, which is a different set from the set that could do work.)

But the connection carries a caveat that this sweep is well placed to make, and it cuts against the easy moral. `policy` is a parameter of `barrier` and of nothing else. Reading the signatures in `src/agentmpi/algorithms.py`, `bcast`, `scatter`, `gather`, `allgather`, `reduce`, `allreduce` and `reduce_scatter` take a `timeout` and no policy at all. The collective that killed `real-tr-p16-full` was an `allreduce`, so no choice among these four names would have saved it; the harness would have had to place a policy-bearing barrier *before* the allreduce, or the protocol would have to extend the policy to the other collectives. That harness did use `PROCEED`, at a barrier in phase 5, downstream of the phase-2 allreduce it never survived to reach. The lesson this sweep contributes is not simply "choose `SHRINK`" but that a failure policy only protects the operation it is attached to, and that in AgentMPI today exactly one operation can carry one.

8 Threats to this reading

The generation diagnosis rests on the fabric, not on the event log, and the event log alone would not support it. No emitted event carries a generation field: `ft.shrink_in_place`'s payload is `{"excluded": [3]}` and nothing more, and `ft.agree`'s payload names voters and excluded but not the `cid` it voted into. I recovered the six-`cid` split by querying `runs/mb-free/faults-shrink/fabric.sqlite` directly, and the inference that each rank's *cached* generation differed — as opposed to some other keying accident — comes from reading `_record_collective` and `shrink_in_place` together with those six rows. It is a tight inference, and the row set (generations 2 through 7 with no 0 or 1, exactly seven voters spread over six `cid`s) admits no other reading I can construct. But if the fabric were unavailable the trace would show only an unexplained 60 s stall, which is itself an argument for putting the generation into the `ft.shrink_in_place` and `ft.agree` payloads.

This is $n = 1$ on a race, and the particular numbers are schedule-dependent. That rank 1 timed out first, that it named six ranks and rank 7 named one, that ranks 5 and 6 landed on the same generation and everyone else did not — all of that is one interleaving of seven processes contending on a SQLite counter. A rerun would produce a different partition of the seven votes across `cid`s and a different split between raisers and vacuous returners. What would *not* change is the structure: seven increments of a shared counter and seven ranks caching it at seven different moments cannot produce a single agreed key except by coincidence, and the failure is therefore a property of the code rather than of this schedule. The counts 14 and 2 in the abstract are this run's; the mechanism is not.

There are no agents here and nothing in this run bears on agent behaviour. `executors: {none: 8}`, zero agent calls, zero tokens. The failure injected is a rank that returns instantly, which models `FAIL_STOP` and only `FAIL_STOP`. `ft.py`'s own taxonomy is explicit that the interesting class for agent systems is `FAIL_PLAUSIBLE` — a rank that returns a confident wrong answer — and that no amount of timeout tuning detects it. Nothing in this sweep, and nothing in the barrier policies at all, addresses that class. A reader taking these four runs as the evidence for AgentMPI's fault-tolerance claims should note that they exercise one row of a six-row table.

The detection latency is a restatement of the deadline, not a measurement of a detector. `detect_p50_s` is 15.196 s against a configured 15.0 s, so the figure is the timeout plus 196 ms of poll granularity and fabric write latency. It says nothing about how quickly AgentMPI *can* notice a failure, because the only detector exercised here is the barrier's own deadline. The lease-based detector in `ft.health`, which is what `ft.py` presents as the failure detector, is never called in this run: `_agree_survivors` would call it, but the harness uses `agree` rather than `shrink`, and `agree` does not. A reader should not generalise 15.2 s into a detection-latency claim.

Similarly, `shrink_p50_s: 0.0` in the published row measures nothing. The benchmark starts its shrink timer *after* the barrier has returned, and by then `shrink_in_place` has already run inside `_central_barrier`. What the 0.0 s times is `comm.refresh()` plus a list comprehension. The actual in-place shrink cost is bounded by the interval between each rank's `ft.declare_failed` and its `ft.shrink_in_place` event, which is under a millisecond for six of the seven survivors and 1 ms for the other — the right order of magnitude, but recovered from the log rather than from the benchmark's instrumentation.

The third defect is read from the source, not observed. That `Communicator._active` is never consumed, and that a second barrier would therefore hang on rank 3 again, follows from grepping `src/agentmpi` and from `comm.size` counting all members. This run never issues a second collective on the shrunken communicator, so the trace neither confirms nor refutes it. It is the one claim here that a targeted experiment — two barriers, one victim — would settle in fifteen seconds, and until someone runs it the claim should be read as a source reading.

Finally, the harness catches its own errors, and that shapes every run-level number. `rank_main` wraps the barrier in `except ampi.AmpiError` and the agree in a second one, so no exception ever reaches `ampi.launch`. That is why `job.finish` says `ok: true` and `failed_ranks: []` for a run containing two `AmpiProcFailed` raises and eight failure rows, why `metrics.json`'s `failed_ranks` is empty, and why `degraded` is `true` only via the incomplete-collective clause. Every judgement above about what went wrong comes from the event log and the fabric; the job-level accounting saw a clean run and would have reported one even if all seven survivors had raised.